VectorCore MMSC

User and Operations Manual

Version 0.1.0a

This document covers installation, configuration, operation, and administration of the VectorCore MMSC. It is intended for end users, laboratory deployments, test setups, and students working with MMS infrastructure.

Table of Contents

1. Introduction
2. Architecture Overview
3. Prerequisites and Installation
4. Configuration Reference
5. Interface Reference
6. Admin UI Guide
7. Billing and CGRateS Integration
8. Operations and Maintenance
9. Troubleshooting
10. Using HAProxy in place of header enrichment

---

1. Introduction

VectorCore MMSC is an open, standards-based Multimedia Messaging Service Centre (MMSC) implementing the OMA MMS 1.3 specification. It supports all major MMS interfaces and is designed to be simple to deploy, configure, and operate in lab, test, and production environments.

What is an MMSC?

An MMSC is the server-side component of the MMS system. When a mobile device sends an MMS message, it is submitted to the MMSC over the MM1 interface. The MMSC stores the message, routes it to the correct destination, and notifies the recipient's device via a WAP push notification. The recipient device then retrieves the message from the MMSC over MM1.

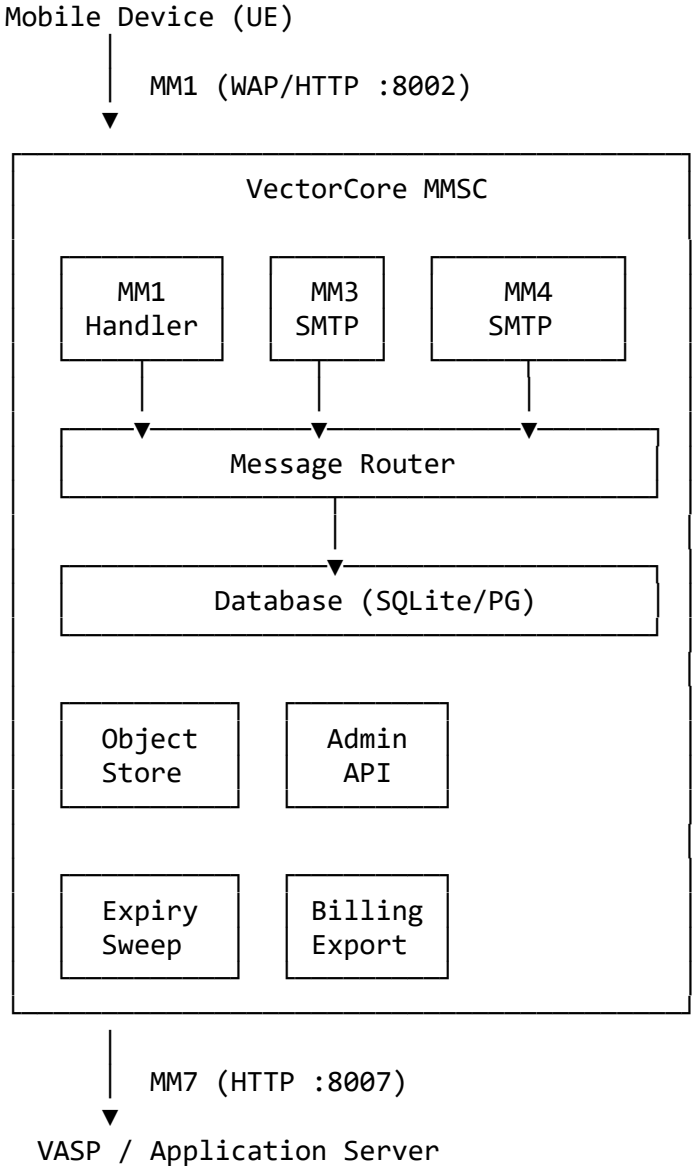
The MMSC also handles inter-operator delivery (MM4), email relay (MM3), and Value-Added Service Provider integration (MM7).

Key Features

- MM1 (WAP/HTTP) — mobile device submission and retrieval
- MM3 — email relay inbound and outbound
- MM4 — inter-MMSC SMTP relay for cross-operator delivery
- MM7 — VASP (Value-Added Service Provider) integration
- WAP Push notification via SMPP
- Per-message expiry and automatic sweep
- CGRateS-compatible CSV CDR billing export
- Media adaptation framework (requires libvips and ffmpeg)
- Web-based administration UI
- SQLite and PostgreSQL database support
- Filesystem, S3, and tiered object storage

---

2. Architecture Overview



Component Summary

3. Prerequisites and Installation

System Requirements

- Linux (x86_64 or arm64)
- 512 MB RAM minimum (1 GB recommended for production)
- SQLite (bundled) or PostgreSQL 13+
- For media adaptation: libvips and ffmpeg installed on the system

Obtaining the Binary

The MMSC is distributed as a single statically linked binary `mmsc`. Copy it to your preferred location, for example `/opt/vectorcore/bin/mmsc`, and make it executable:

```
chmod +x /opt/vectorcore/bin/mmsc
```

Directory Structure

Create the following directories before starting the service:

```
mkdir -p /opt/vectorcore/data/store
mkdir -p /opt/vectorcore/data/billing
mkdir -p /opt/vectorcore/log
```

Database Setup

SQLite (recommended for lab and test)

No setup required. The database file is created automatically on first start. Set the DSN to a file path:

```
database:
  driver: sqlite
  dsn: "./vectorcore-mmsc.db"
```

PostgreSQL

Create a database and user:

```
CREATE DATABASE mmsc;
CREATE USER mmsc WITH PASSWORD 'yourpassword';
GRANT ALL PRIVILEGES ON DATABASE mmsc TO mmsc;
```

Set the DSN:

```
database:
  driver: postgres
  dsn: "postgres://mmsc:yourpassword@localhost:5432/mmsc?sslmode=disable"
```

Database schema migrations run automatically on startup.

Starting the Service

```
mmsc -c /path/to/config.yaml
```

Available flags:

Flag	Description
`-c`, `--config-file`	Path to configuration file (default: `config.yaml`)
`-d`	Enable debug logging to console
`-v`	Print version and exit

Running as a systemd Service

Create `/etc/systemd/system/mmsc.service`:

```
[Unit]
Description=VectorCore MMSC
After=network.target

[Service]
ExecStart=/opt/vectorcore/bin/mmsc -c /opt/vectorcore/config.yaml
WorkingDirectory=/opt/vectorcore
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Enable and start:

```
systemctl daemon-reload
systemctl enable mmsc
systemctl start mmsc
```

---

4. Configuration Reference

All configuration is in a single YAML file. The sections below document every available key.

database

Controls the database connection.

Key	Type	Default	Description
`driver`	string	`postgres`	Database driver: `sqlite` or `postgres`

Key	Type	Default	Description
`dsn`	string	—	Database connection string or file path
`max_open_conns`	int	`20`	Maximum open database connections
`max_idle_conns`	int	`5`	Maximum idle database connections
`runtime_reload_interval`	duration	`5s`	How often runtime configuration (peers, VASPs, etc.) is reloaded from the database

mm1

Controls the MM1 WAP/HTTP interface used by mobile devices.

Key	Type	Default	Description
`listen`	string	`:8002`	Address and port to listen on
`retrieve_base_url`	string	—	Base URL used in WAP push notifications for MT message retrieval. Must be reachable by the UE. Example: `http://mmsc.example.com:8002/mms/retrieve`
`max_body_size_bytes`	int	`10485760`	Maximum HTTP body size accepted. Acts as a first-pass limit before PDU decoding.

mm3

Controls the MM3 email relay interface.

Key	Type	Default	Description
`inbound_listen`	string	`:2026`	SMTP listen address for inbound email

Key	Type	Default	Description
`max_message_size_bytes`	int	`10485760`	Maximum inbound email size
`tls_cert_file`	string	—	Path to TLS certificate file
`tls_key_file`	string	—	Path to TLS private key file

mm4

Controls the MM4 inter-MMSC SMTP interface.

Key	Type	Default	Description
`inbound_listen`	string	`:2025`	SMTP listen address for inbound MM4
`hostname`	string	—	This MMSC's hostname, used in SMTP EHLO and as the node identifier
`max_message_size_bytes`	int	`10485760`	Maximum inbound MM4 message size
`tls_cert_file`	string	—	Path to TLS certificate file
`tls_key_file`	string	—	Path to TLS private key file

mm7

Controls the MM7 VASP interface.

Key	Type	Default	Description
`listen`	string	`:8007`	HTTP listen address
`path`	string	`/mm7`	URL path for MM7 SOAP endpoint
`eaif_path`	string	`/eaif`	URL path for EAIF endpoint
`version`	string	`5.3.0`	MM7 protocol version advertised
`eaif_version`	string	`3.0`	EAIF version advertised
`namespace`	string	3GPP namespace URI	XML namespace used in MM7 SOAP envelopes

api

Controls the Admin REST API and UI.

Key	Type	Default	Description
`listen`	string	`:8080`	Address and port for the Admin API and UI

store

Controls where MMS payload files are stored.

Key	Type	Default	Description
`backend`	string	`filesystem`	Storage backend: `filesystem`, `s3`, or `tiered`

store.filesystem

Key	Type	Default	Description
`root`	string	—	Root directory for stored files

store.s3

Key	Type	Default	Description
`endpoint`	string	—	S3 endpoint URL
`bucket`	string	—	S3 bucket name
`access_key`	string	—	S3 access key
`secret_key`	string	—	S3 secret key
`region`	string	—	S3 region

store.tiered

Uses local filesystem as a cache with S3 as the primary store. Requires both `filesystem` and `s3` to be configured.

Key	Type	Default	Description
`offload_after`	duration	`1h`	How long before a file is moved from local cache to S3
`local_cache`	bool	`true`	Whether to cache files locally before offloading

adapt

Controls media adaptation. Requires libvips and ffmpeg installed on the system.

Key	Type	Default	Description
`enabled`	bool	`false`	Enable media adaptation pipeline
`libvips_path`	string	`/usr/bin/vips`	Path to the vips binary
`ffmpeg_path`	string	`/usr/bin/ffmpeg`	Path to the ffmpeg binary

When disabled, the Adaptation page in the Admin UI will display a warning and class definitions will be stored but not applied.

limits

Controls system-wide message size and retention limits.

Key	Type	Default	Description
`max_message_size_bytes`	int	`5242880`	Global ingress limit (5 MB). Messages larger than this are rejected at MM1 with OMA error code 0x93 (Error-permanent-message-size-exceeded).
`default_message_expiry`	duration	`168h`	Expiry applied to messages that carry no explicit expiry in the original PDU (7 days).
`max_message_retention`	duration	`720h`	Hard ceiling on message retention (30 days). Even if a sender supplies a longer expiry it is clamped to this value.

billing

Controls CDR export for CGRateS integration.

Key	Type	Default	Description
`enabled`	bool	`false`	Enable CDR CSV export

Key	Type	Default	Description
`export_dir`	string	`./data/billing`	Directory where CSV files are written. Point this at the directory watched by CGRateS `cdrc`.
`interval`	duration	`1h`	How often the export runs
`tenant`	string	`cgrates.org`	CGRateS tenant string written into every CDR row
`req_type`	string	`*postpaid`	CGRateS request type: `*postpaid` or `*prepaid`
`node_id`	string	`mm4.hostname`	Node identifier written into the `orighost` CDR field. Defaults to `mm4.hostname` if empty.

log

Controls logging output.

Key	Type	Default	Description
`level`	string	`info`	Log level: `debug`, `info`, `warn`, `error`
`format`	string	`json`	Log format: `json` or `console`
`file`	string	`./log/mmsc.log`	Path to log file

---

5. Interface Reference

MM1 — Mobile Device Interface

MM1 is the WAP/HTTP interface between the MMSC and mobile devices (UEs). It runs on port 8002 by default.

Message Submission (MO)

The UE submits an `m-send-req` PDU encoded in WAP binary format with Content-Type `application/vnd.wap.mms-message`. The MMSC responds with an `m-send-conf` PDU.

If the message exceeds `limits.max_message_size_bytes`, the MMSC returns an `m-send-conf` with response status `Error-permanent-message-size-exceeded` (OMA token 0x93). The UE will not retry.

Endpoint: `POST /mms/send`

Message Retrieval (MT)

The UE is notified of a pending message via a WAP push over SMPP. The WAP push contains an `m-notification-ind` PDU with a `Content-Location` URL. The UE fetches that URL to retrieve the message.

Endpoint: `GET /mms/retrieve?id={message_id}`

The response is an `m-retrieve-conf` PDU. SMIL presentation parts are stripped before delivery and the content is wrapped in `application/vnd.wap.multipart.mixed` for correct rendering on iOS and Android.

Delivery Report (m-notify-resp)

After retrieving a message the UE sends an `m-notify-resp` PDU indicating whether it accepted or rejected the message. The MMSC uses this to update message status.

Endpoint: `POST /mms/notify`

SMPP for WAP Push

The MMSC delivers WAP push notifications via an SMPP upstream. Configure one or more SMPP upstreams in the Admin UI under the SMPP section. The MMSC will submit a WAP push to each recipient's MSISDN through the configured upstream.

The UE's MSISDN is extracted from HTTP headers supplied by the carrier network: - `X-WAP-Network-Client-MSISDN` - `X-MSISDN` - `X-Nokia-MSISDN`

MM3 — Email Relay

MM3 relays MMS messages to and from email systems. The inbound SMTP server listens on port 2026 by default.

Inbound email messages are decoded from MIME, converted to MMS parts, and routed through the normal message pipeline. Configure the MM3 relay settings in the Admin UI under MM3.

MM4 — Inter-MMSC Relay

MM4 is used to exchange MMS messages between MMSCs at different operators. It uses SMTP as the transport with MMS PDUs encoded as MIME attachments.

Inbound: The MMSC listens for SMTP connections on port 2025. Configure allowed peers in the Admin UI under Peers.

Outbound: When a message is addressed to a recipient on a remote operator, the router selects an MM4 peer and relays the message via SMTP.

MM7 — VASP Interface

MM7 allows application servers and Value-Added Service Providers to submit MMS messages. Two protocol variants are supported:

- **MM7 SOAP** — standard 3GPP MM7 SOAP/XML (`POST /mm7`)
- **EAIF** — Extended Application Interface (`POST /eaif`)

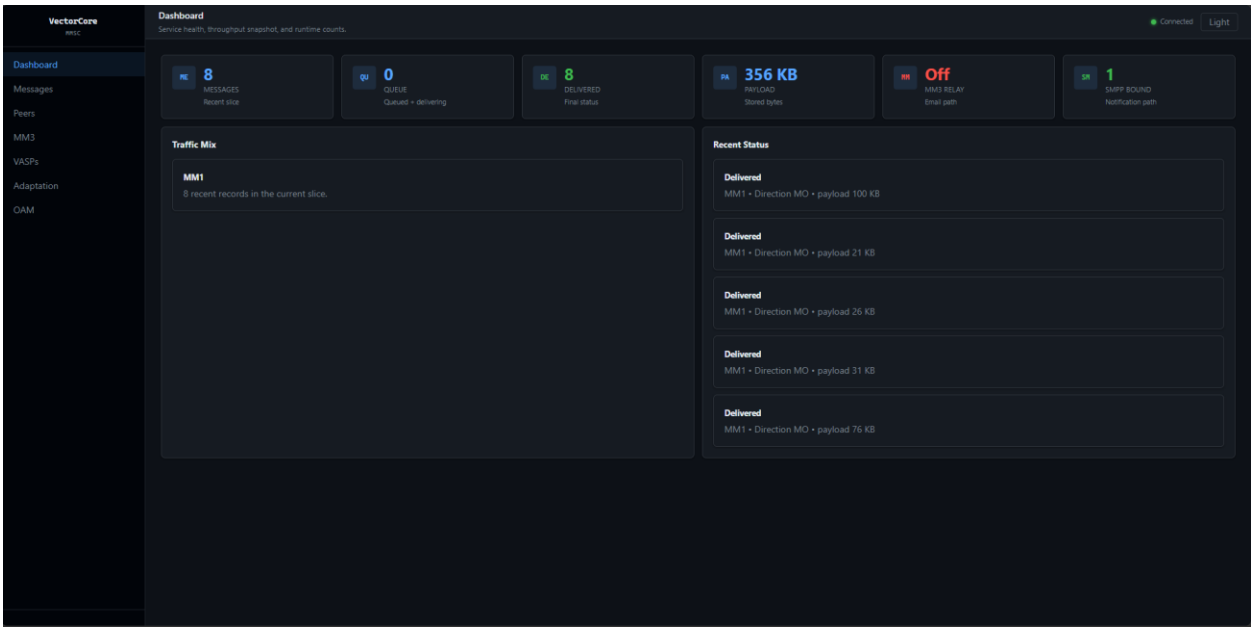
Configure VASPs in the Admin UI under VASPs. Each VASP has its own credentials, allowed IPs, and delivery callback URL.

---

6. Admin UI Guide

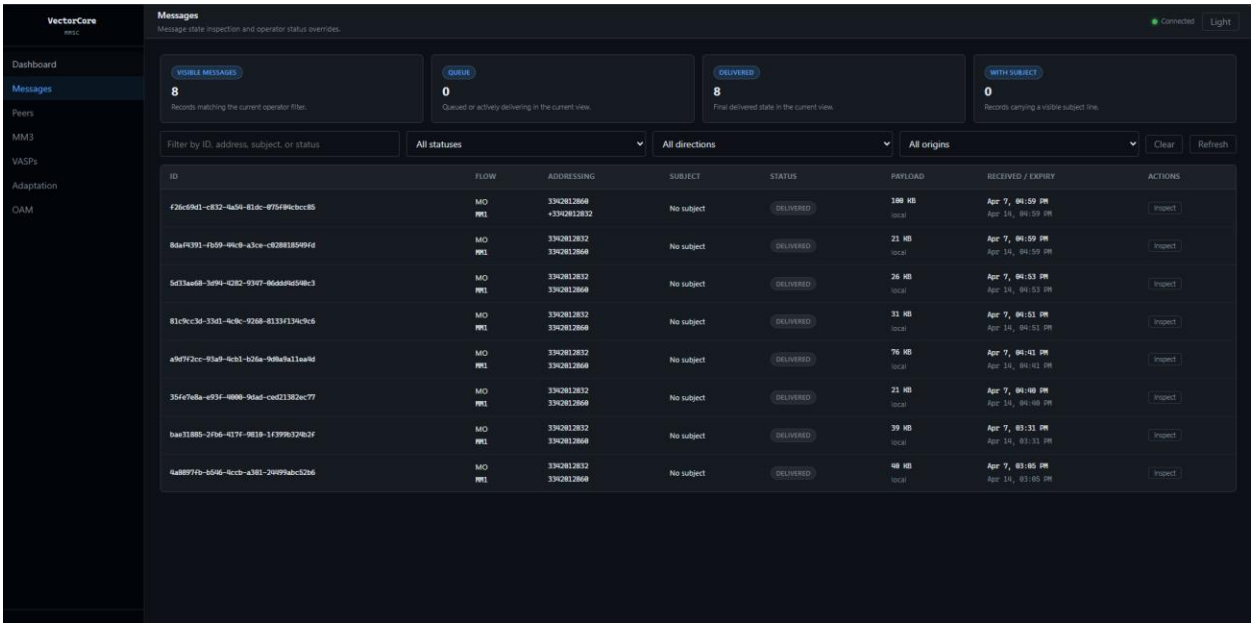
The Admin UI is served at `http://<host>:8080` and provides a web interface for operating the MMSC. It updates automatically every 15 seconds.

Dashboard



Displays a real-time snapshot of service health including: - Queue depth (queued and delivering messages) - Total delivered and forwarded counts - SMPP upstream connection status - Service uptime and version

Messages



Displays all messages in the system with filtering by status, direction, and interface. Each row shows:

Column	Description
ID	Internal message UUID
Flow	Direction (MO/MT) and originating interface (MM1, MM4, MM3, MM7)
Addressing	From and To addresses
Subject	Message subject if present
Status	Current lifecycle status
Payload	Message size and origin host
Received / Expiry	When the message was received and when it will be deleted

Message Status Values

Status	Description
Queued	Awaiting delivery
Delivering	WAP push sent, waiting for UE retrieval
Delivered	UE has retrieved the message
Expired	Expiry time passed before delivery
Rejected	UE explicitly rejected the message
Forwarded	Relayed to MM4 or MM3 peer
Unreachable	Delivery failed, no further retries

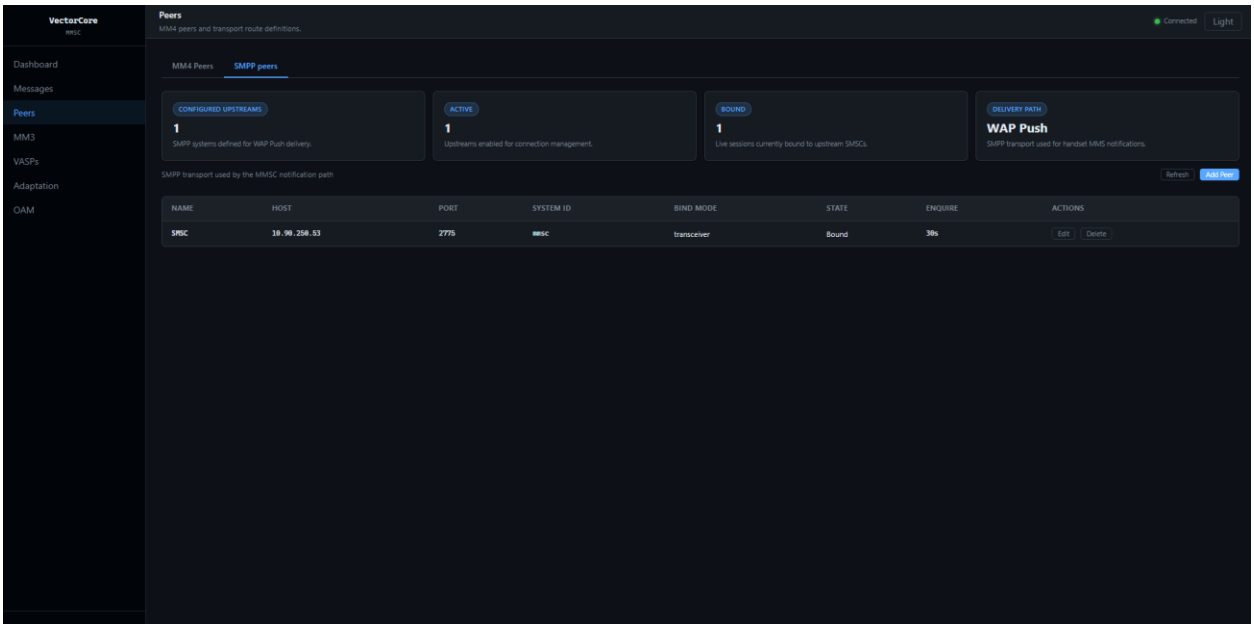
Inspect Modal

Clicking **Inspect** on any message opens a detailed view showing: - Full addressing and metadata - Operator summary and recommended next action - Message event trail (all state changes, operator notes) - SMPP delivery trail (submission attempts and outcomes) - Inter-MMSC / VASP path information

Operator Actions (within Inspect)

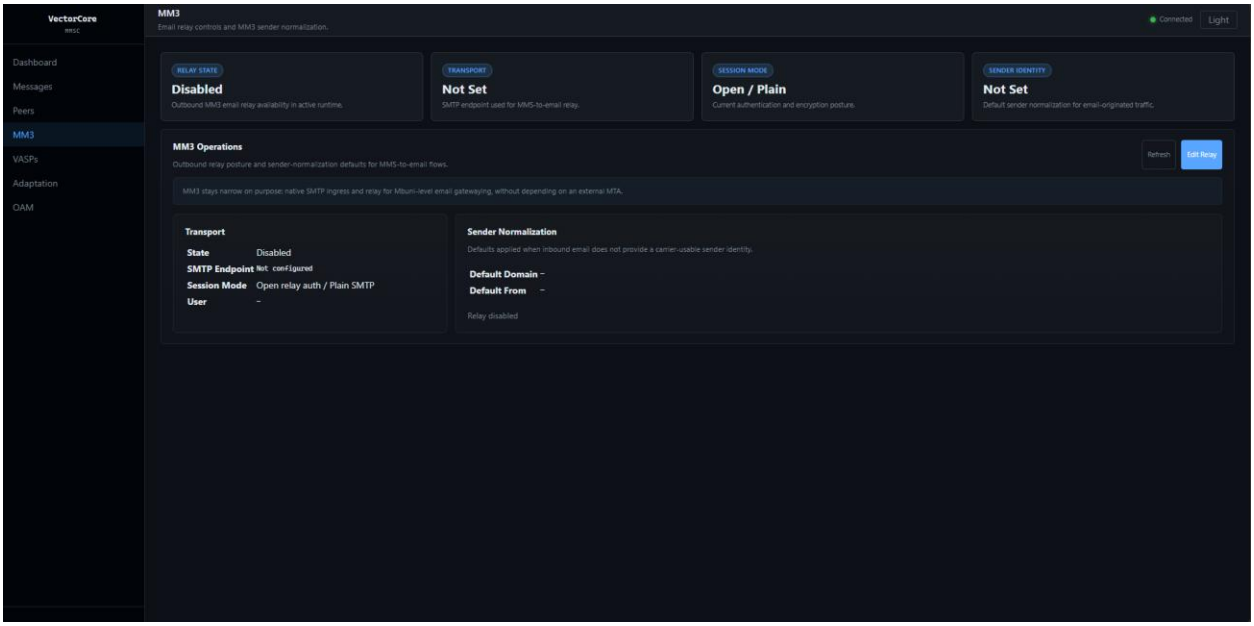
Action	Description
Add Note	Attach a free-text note to the message event trail
Requeue	Reset status to Queued to trigger a fresh delivery attempt
Delete	Remove the message from the queue (only available for Queued/Delivering)
Mark Delivered / Rejected / Unreachable	Force a terminal status override

Peers



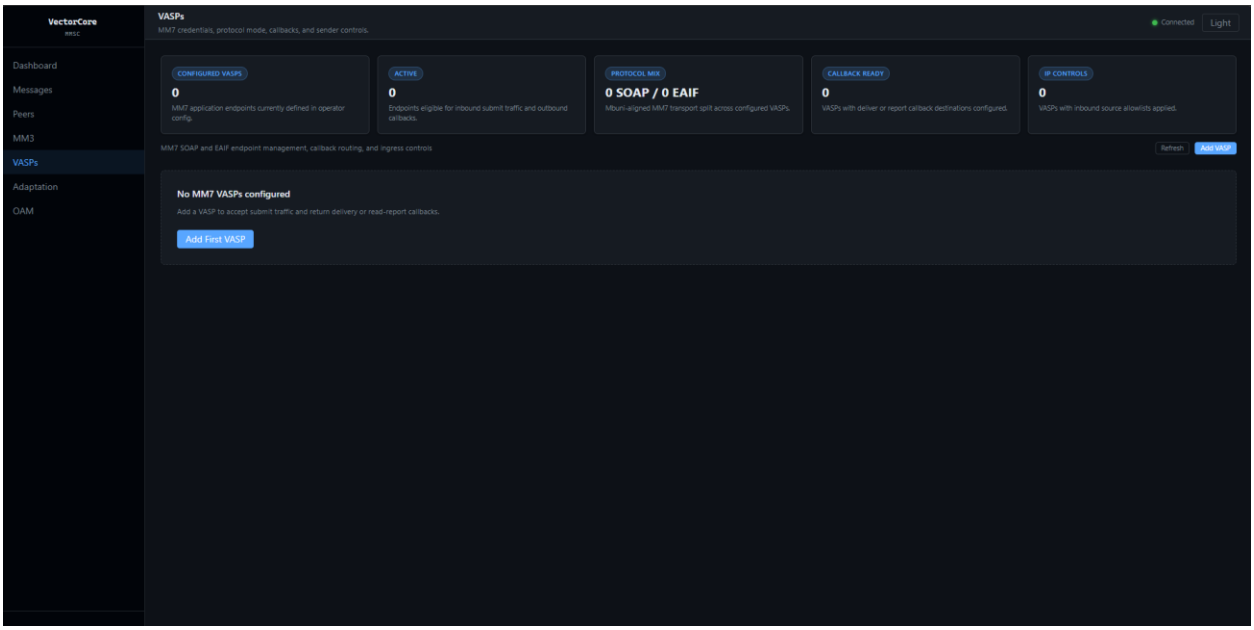
Manage MM4 inter-MMSC peers. Each peer defines an SMTP host, port, TLS settings, and allowed source IPs for inbound connections.

MM3



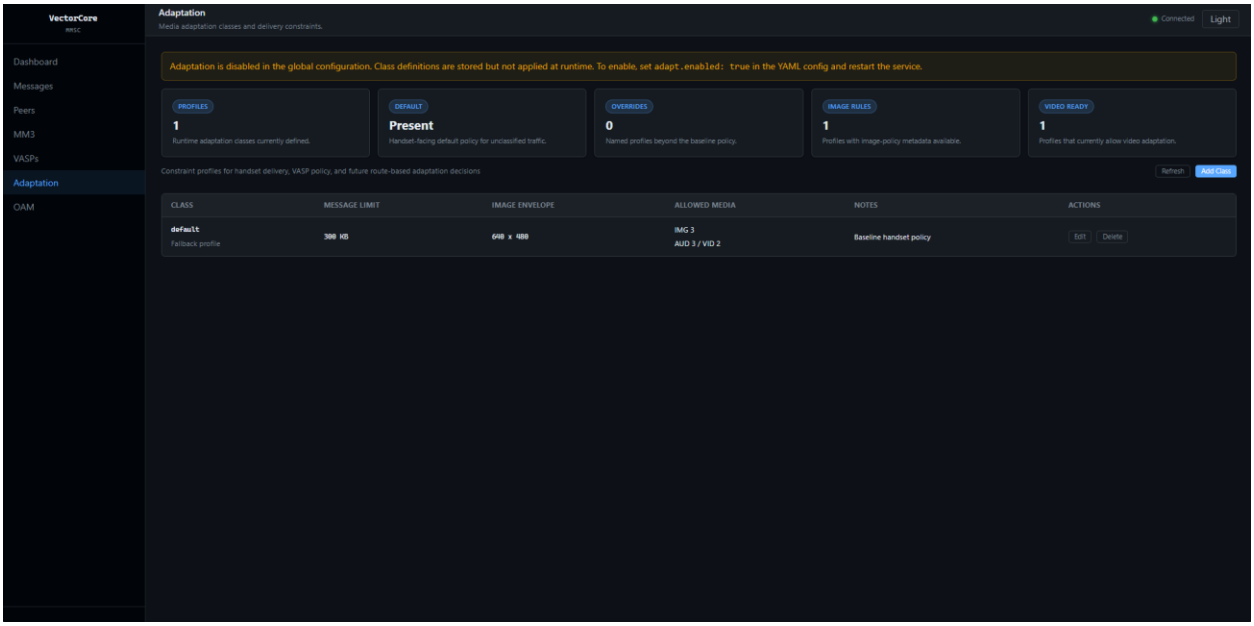
Configure the outbound email relay. When enabled, messages destined for email addresses are relayed via the configured SMTP server.

VASPs



Manage MM7 Value-Added Service Provider credentials. Each VASP entry defines: - Protocol (MM7 SOAP or EAIF) - Shared secret for authentication - Allowed source IPs - Delivery and report callback URLs - Maximum message size

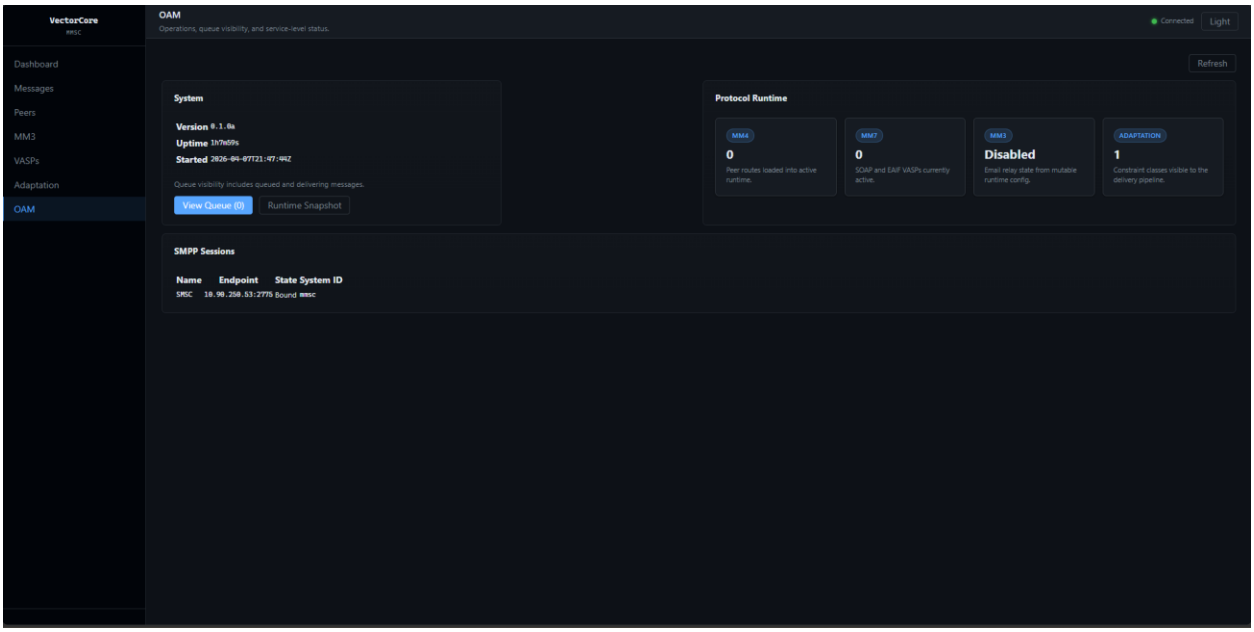
Adaptation



Manage media adaptation class profiles. Each class defines size limits and allowed media types for a group of subscribers or routes.

> **Note:** If `adapt.enabled` is `false` in the configuration, a warning is displayed and class definitions are stored but not applied at runtime.

OAM



Operations, Administration, and Maintenance view showing: - Queue visibility counts - Per-status message totals - Service uptime and version

7. Billing and CGRateS Integration

VectorCore MMSC exports CDR (Charging Data Record) files in CSV format compatible with the CGRateS Open Charging System. Files are written to the configured `billing.export_dir` on the configured `billing.interval`.

Enabling Billing Export

Set `billing.enabled: true` in `config.yaml` and configure the export directory to match the directory watched by CGRateS `cdrc`:

```yaml
billing:
  enabled: true
  export_dir: "/var/spool/cgrates/cdrc/in"
  interval: 1h
  tenant: "cgrates.org"
  req_type: "*postpaid"
  node_id: "mmsc.example.com"
```

CDR File Format

Files are named:

`MMSC_CDR_{node_id}_{YYYYMMDD}_{HHMM}_{sequence}.csv`

Example:

`MMSC_CDR_mmsc.example.com_20260407_1400_0001.csv`

Each file contains a header row followed by one row per message:

Column	CGRateS Field	Description
`cgrid`	cgrid	Message UUID
`tor`	tor	Always `*mms`
`accid`	accid	Transaction ID
`orighost`	orighost	Node ID (mm4.hostname)
`reqtype`	reqtype	`*postpaid` or `*prepaid`
`tenant`	tenant	CGRateS tenant
`category`	category	Always `mms`
`account`	account	Originating MSISDN (who to charge)
`destination`	destination	Recipient address(es), semicolon separated
`setuptime`	setuptime	Message received timestamp (RFC3339)

Column	CGRateS Field	Description
`answertime`	answertime	Delivery timestamp if delivered, empty otherwise
`usage`	usage	Message size in bytes
`cost`	cost	Empty — CGRateS calculates this from tariff plans
`direction`	direction	`out` for MO, `in` for MT

Watermark

The exporter maintains a `.watermark` file in the export directory tracking the last exported record timestamp. This ensures: - No records are exported twice after a restart - No records are skipped between export runs

CGRateS cdrc Configuration

In your CGRateS configuration, point a `cdrc` instance at the export directory:

```
"cdrc": [{
  "id": "mmsc",
  "enabled": true,
  "cdrs_conns": ["*internal"],
  "cdr_in_dir": "/var/spool/cgrates/cdrc/in",
  "cdr_out_dir": "/var/spool/cgrates/cdrc/out",
  "cdr_source_id": "mmsc",
  "data_usage_multiply_factor": 1,
  "cdr_filter": "",
  "fields": [
    {"tag": "cgrid",       "field_id": "CGRID",       "type": "*composed", "v
alue": "~*req.0"},
    {"tag": "tor",         "field_id": "ToR",         "type": "*composed", "v
alue": "~*req.1"},
    {"tag": "accid",       "field_id": "OriginID",    "type": "*composed", "v
alue": "~*req.2"},
    {"tag": "orighost",    "field_id": "OriginHost",  "type": "*composed", "v
alue": "~*req.3"},
    {"tag": "reqtype",     "field_id": "RequestType", "type": "*composed", "v
alue": "~*req.4"},
    {"tag": "tenant",      "field_id": "Tenant",      "type": "*composed", "v
alue": "~*req.5"},
    {"tag": "category",    "field_id": "Category",    "type": "*composed", "v
alue": "~*req.6"},
    {"tag": "account",     "field_id": "Account",     "type": "*composed", "v
alue": "~*req.7"},
    {"tag": "destination", "field_id": "Destination", "type": "*composed", "v
alue": "~*req.8"},
    {"tag": "setuptime",   "field_id": "SetupTime",   "type": "*composed", "v
```

```
alue": "~*req.9"},
    {"tag": "answertime",  "field_id": "AnswerTime",  "type": "*composed", "v
alue": "~*req.10"},
    {"tag": "usage",       "field_id": "Usage",       "type": "*composed", "v
alue": "~*req.11"},
    {"tag": "cost",        "field_id": "Cost",        "type": "*composed", "v
alue": "~*req.12"}
  ]
}]
```

---

8. Operations and Maintenance

Message Lifecycle

```
Submit (MO) → Queued → Delivering → Delivered
                     ↘ Rejected
                     ↘ Unreachable
                     ↘ Expired (sweep)
```

Message Expiry and Automatic Cleanup

The MMSC enforces message expiry automatically. At ingestion, every message is assigned an expiry timestamp:

- If the originating PDU contains an explicit X-Mms-Expiry header, that value is used.
- If no expiry is present, `limits.default_message_expiry` (default 7 days) is applied from the received timestamp.
- In all cases, the expiry is capped at `limits.max_message_retention` (default 30 days) from the received timestamp.

A background sweep process runs every minute and permanently removes all messages whose expiry has passed — deleting both the database record and the stored payload file. Empty store directories are pruned automatically.

The stored expiry is the single point of truth. The sweep query is simply:

```
SELECT * FROM messages WHERE expiry < now()
```

Log Files

Logs are written in JSON format to the path configured in `log.file`. Log level can be set to `debug`, `info`, `warn`, or `error`.

To enable debug logging at runtime without restarting, pass `-d` at startup.

Example log entries:

```
{"level":"info","ts":"2026-04-07T14:00:00Z","msg":"mm1 mo accepted","interfac
e":"mm1","message_id":"f9d252f7-...","response_bytes":45}
{"level":"info","ts":"2026-04-07T14:00:01Z","msg":"sweep: purging expired mes
```

sages","count":3}
{"level":"info","ts":"2026-04-07T14:00:00Z","msg":"billing: CDR file written"
,"file":"MMSC_CDR_mmsc_20260407_1400_0001.csv","records":142}

Database Maintenance

For PostgreSQL deployments, standard maintenance applies — vacuuming, index maintenance, and regular backups. The messages table is the highest-churn table; the expiry sweep handles row deletion automatically.

For SQLite deployments, periodic `VACUUM` is recommended after large sweep runs to reclaim disk space:

```
VACUUM;
```

Checking Service Health

```
curl http://localhost:8080/healthz
curl http://localhost:8080/readyz
```

The `/readyz` endpoint returns non-200 if the database is unreachable.

Upgrading

1. Stop the service
2. Replace the `mmsc` binary
3. Start the service — schema migrations run automatically

---

9. Troubleshooting

UE Receives No Notification

1. Check SMPP upstream status in the Admin UI under the Dashboard or SMPP section — the upstream must show **Bound**.
2. Verify `mm1.retrieve_base_url` is reachable from the UE's network.
3. Check that the UE's MSISDN is being passed in the HTTP headers (`X-WAP-Network-Client-MSISDN`, `X-MSISDN`, or `X-Nokia-MSISDN`) from the carrier gateway.
4. Confirm MMS is enabled on the UE's SIM/APN settings.
5. Check the message event trail in the Inspect modal for dispatch errors.

UE Shows "Attachment" Instead of Inline Image

This is typically a WAP binary encoding issue. Verify: - The MMSC version is 0.1.0a or later — earlier versions had incorrect WAP token encoding. - The message is delivered as `application/vnd.wap.multipart.mixed` (token 0xA3).

Message Stuck in Delivering

The UE received the WAP push but has not retrieved the message. This can happen if: - The `mm1.retrieve_base_url` is not reachable from the UE. - The UE's MMS APN is not configured correctly. - The UE dismissed the notification before retrieving.

Use **Requeue** in the Inspect modal to trigger a fresh WAP push.

Message Status Shows Rejected

The UE sent an `m-notify-resp` indicating it rejected the message. Common causes: - Message too large for the UE. - Unsupported content type. - UE storage full.

Check the message event trail for the raw status token received.

MM4 Delivery Failing

1. Verify the peer's domain is configured in the Admin UI under Peers.
2. Check that the peer's SMTP host is reachable from the MMSC server.
3. Confirm TLS settings match what the remote MMSC expects.
4. Review logs for SMTP error responses.

Large Messages Rejected

If the UE receives a permanent rejection error when sending: - Check `limits.max_message_size_bytes` in the configuration. - The default limit is 5 MB. Increase if needed. - Note that `mm1.max_body_size_bytes` is a separate HTTP-level limit and should be set higher than `limits.max_message_size_bytes`.

Billing Files Not Appearing

1. Confirm `billing.enabled: true` in the configuration.
2. Check that `billing.export_dir` exists and is writable by the mmsc process.
3. The exporter runs on the configured `billing.interval` — no files will appear until the first interval elapses after startup.
4. Check logs for `billing:` entries indicating export status or errors.

Checking Version

```
mmsc -v
```

Or via the API:

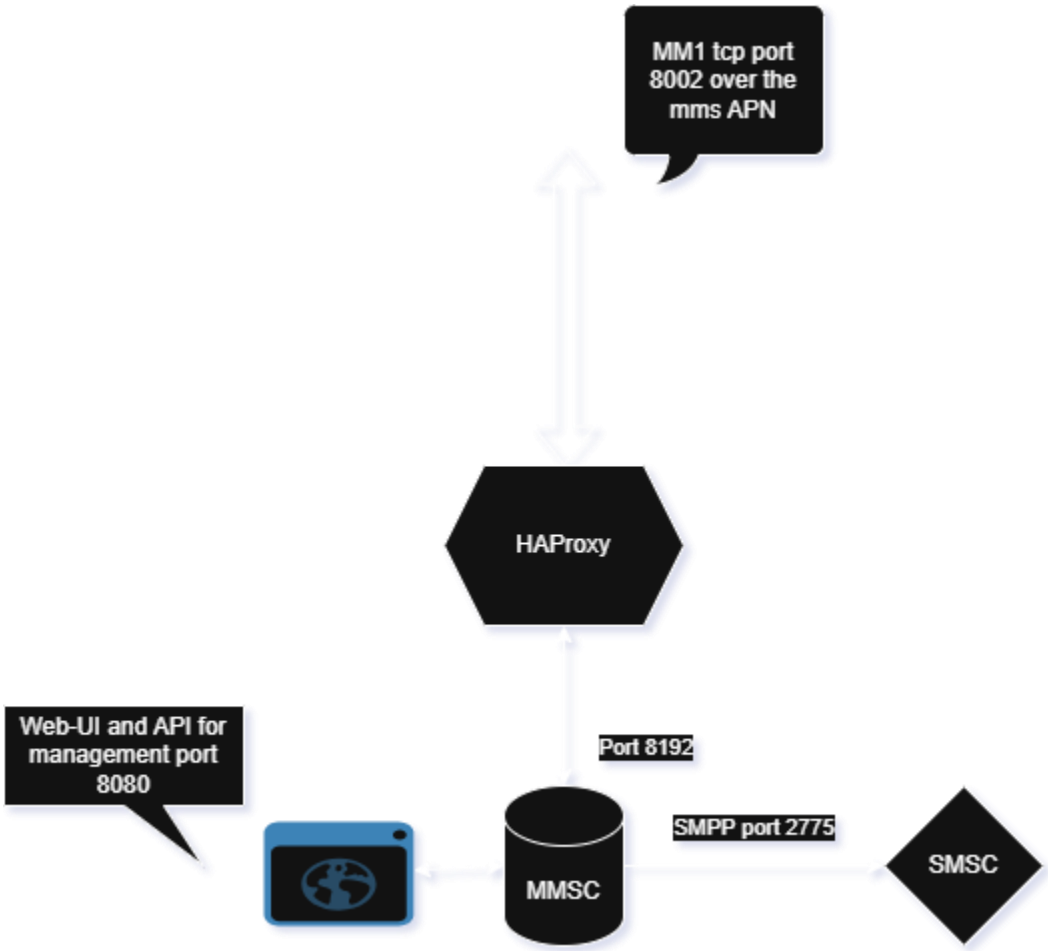
```
curl http://localhost:8080/api/v1/system/status | jq .body.version
```

---

10. Using HAProxy in place of header enrichment

Header enrichment inserts the sender identity into the HTTP request, typically at the **GGSN/PGW or WAP proxy layer**: X-WAP-Network-Client-MSISDN: 15551234567

Most Open source PGW/UPF do not support this feature and it's a licensed feature on most commercial Cores. A simple fix for testing is to use HAProxy to insert the sender identity for us.



Sample haproxy.conf:

```
global
        log /dev/log    local0
        log /dev/log    local1 notice
        chroot /var/lib/haproxy
        stats socket /run/haproxy/admin.sock mode 660 level admin expose-fd listeners
        stats timeout 30s
        user haproxy
        group haproxy
        daemon

        # Default SSL material locations
        ca-base /etc/ssl/certs
```

```
        crt-base /etc/ssl/private

        # See: https://ssl-config.mozilla.org/#server=haproxy&server-version=2.0.3&config=intermediate
        ssl-default-bind-ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECDHE-ECDSA-
AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:ECDHE-ECDSA-CHACHA20-POLY1305:ECDHE-RSA-CHACHA20-
POLY1305:DHE-RSA-AES128-GCM-SHA256:DHE-RSA-AES256-GCM-SHA384
        ssl-default-bind-ciphersuites
TLS_AES_128_GCM_SHA256:TLS_AES_256_GCM_SHA384:TLS_CHACHA20_POLY1305_SHA256
        ssl-default-bind-options ssl-min-ver TLSv1.2 no-tls-tickets

defaults
        log     global
        mode    http
        option  httplog
        option  dontlognull
        timeout connect 5000
        timeout client  50000
        timeout server  50000
        errorfile 400 /etc/haproxy/errors/400.http
        errorfile 403 /etc/haproxy/errors/403.http
        errorfile 408 /etc/haproxy/errors/408.http
        errorfile 500 /etc/haproxy/errors/500.http
        errorfile 502 /etc/haproxy/errors/502.http
        errorfile 503 /etc/haproxy/errors/503.http
        errorfile 504 /etc/haproxy/errors/504.http


frontend mms
    bind :8002

    # Match /mms/retrieve
    acl is_retrieve path /mms/retrieve

    # Add header only if NOT retrieve
    http-request add-header X-WAP-Network-Client-MSISDN %[path,field(2,/)] if !is_retrieve

    default_backend mms


backend mms
    mode http
    option forwardfor
    server localhost 127.0.0.1:8191
```

Then under your mms APN settings set the MMSC to http://{MMSC_IP}:8002/{UE_MSISDN}
Where {MMSC_IP} is the IP address of your MMSC and {UE_MSISCN} is the MSISDN of the local UE